

Google Antigravity 2.5.0

Manual Oficial de Referencia, Manejo y Arquitectura Agentica

Documentación técnica exhaustiva para desarrolladores, arquitectura de sistemas, superficies de interacción, SDK de Python, comandos slash y modelo de seguridad.

Índice de Contenidos

1. Introducción y Filosofía AI-First
2. Superficies de Interacción (IDE, Desktop 2.0, CLI, SDK)
3. Catálogo de Herramientas Nativas (Tooling)
4. Sistema de Extensión: Skills, Rules, Plugins y MCP
5. Comandos Slash (/) y Menú de Menciones (@)
6. Seguridad, Sandbox y Control de Permisos
7. Guía de Integración Práctica y Casos de Uso

1. Introducción y Filosofía AI-First

Google Antigravity (AGY) es la plataforma de desarrollo agéntico de nueva generación diseñada por el equipo de Advanced Agentic Coding en Google DeepMind. Antigravity transforma la relación tradicional entre el programador y el ordenador, pasando de una programación instructiva pasiva a un modelo de **Pair Programming Autónomo** de alta fidelidad.

Concepto Clave: Antigravity no es simplemente un asistente de chat o un autocompletador de código; es un entorno donde el agente cuenta con capacidad de razonamiento multi-paso, lectura e inspección de código fuente, ejecución de pruebas en la terminal, navegación web y toma de decisiones contextuales.

Principales Modos de Interacción

- **Pasivo (Tab Autocomplete / Supercomplete):** Predicción continua del siguiente cambio de código a nivel de línea o diff completo, activado con un solo toque de teclado (Tab).
- **Instructivo (Inline Edit - Ctrl+I / ⌘+I):** Ediciones localizadas y enfocadas sobre bloques de código seleccionados o generación puntual de código en el cursor.
- **Colaborativo (Agent Mode):** Pareja de programación completa capaz de ejecutar planes de varios pasos, diagnosticar errores de compilación, editar múltiples archivos y verificar su trabajo mediante comandos de consola.

2. Superficies de Interacción

Antigravity 2.5.0 proporciona 4 entornos principales diseñados para adaptarse a cualquier flujo de trabajo de ingeniería de software:

2.1. Antigravity IDE

Editor de código standalone AI-first (basado en la arquitectura VS Code). Proporciona la integración más profunda con el código fuente en tiempo real:

- **Inline Code Lenses:** Botones de acción directa sobre funciones y clases (ej: "Refactorizar", "Generar Test", "Explicar Código").
- **Visual Diff Overlays:** Indicadores visuales en verde y rojo integrados en el editor para revisar y aceptar/rechazar cambios directamente sobre el lienzo de código.
- **Diagnostic Auto-Fix:** Corrección automática de errores detectados por el linter o el compilador con un solo clic.

2.2. Antigravity 2.0 (Aplicación de Escritorio)

Aplicación de escritorio Electron independiente que permite orquestar agentes sin depender de un editor abierto. Incluye:

- **Chat Canvas:** Panel central conversacional con soporte de comandos slash, menciones y carga de imágenes.
- **Auxiliary Pane (HTML):** Panel lateral que muestra subagentes activos, tareas programadas, archivos modificados y vista previa de artefactos.

2.3. Antigravity CLI (`agy`)

Interfaz de línea de comandos ultrarrápida para terminales Linux, macOS y Windows (PowerShell):

```
# Iniciar la interfaz interactiva en terminal
agy

# Consultar ayuda y banderas
agy --help

# Configuración global en:
# ~/.gemini/antigravity-cli/settings.json
```

2.4. Antigravity Python SDK (`google-antigravity`)

Biblioteca oficial en PyPI para integrar y orquestar agentes en scripts de Python o pipelines de CI/CD:

```
pip install google-antigravity
```

```
import asyncio
import sys
from google.antigravity import Agent, LocalAgentConfig, CapabilitiesConfig

async def main():
    # Configuración con permisos de escritura y comandos habilitados
    config = LocalAgentConfig(
        system_instructions="Eres un asistente experto en automatización.",
        capabilities=CapabilitiesConfig()
    )

    async with Agent(config) as agent:
        response = await agent.chat("Analiza el archivo main.py y corrige los errores.")
        async for token in response:
            sys.stdout.write(token)
            sys.stdout.flush()

if __name__ == "__main__":
    asyncio.run(main())
```

3. Catálogo de Herramientas Nativas (Tooling)

El núcleo de AntigraVity cuenta con herramientas nativas fuertemente tipadas que el agente ejecuta de forma autónoma:

Herramienta	Propósito Principal	Modo de Operación
<code>view_file</code>	Inspección y lectura de archivos de código o binarios.	Lectura por rangos de líneas o offset.
<code>replace_file_content</code>	Edición exacta de un bloque contiguo de texto en un archivo.	Reemplazo atómico con diff.
<code>multi_replace_file_content</code>	Edición simultánea de múltiples bloques no adyacentes.	Reemplazo por chunks múltiples.
<code>run_command</code>	Ejecución de comandos en la consola del sistema operativo.	Síncrono o asíncrono (Task).
<code>invoke_subagent</code>	Lanzamiento de subagentes especializados en segundo plano.	Concurrente / Multi-hilo.
<code>schedule</code>	Programación de temporizadores únicos o tareas cron recurrentes.	Segundo plano / Notificación.
<code>search_web</code>	Búsqueda de información y documentación en la web pública.	Búsqueda HTTP con resumen.
<code>generate_image</code>	Generación o edición de imágenes e interfaces de usuario.	Creación de recursos visuales.

4. Sistema de Extensión: Skills, Rules, Plugins y MCP

4.1. Skills (Habilidades)

Las habilidades son paquetes reutilizables guardados en carpetas con un archivo `SKILL.md` central que define instrucciones, scripts y ejemplos para una tarea específica (ej: consulta a bases de datos científicas, automatizaciones de Android CLI, etc.).

4.2. Rules (Reglas)

Ubicadas en `.agents/rules/` o en la configuración global. Permiten definir normas de obligado cumplimiento, como guías de estilo, prohibición de commits directos en `main` o requisitos de testing.

4.3. Model Context Protocol (MCP)

Antigravity 2.5.0 soporta nativamente el protocolo abierto MCP, lo que permite conectar el agente a servidores MCP de terceros para interactuar con bases de datos PostgreSQL, APIs de Slack, Figma o Jira.

5. Comandos Slash (/) y Menciones (@)

5.1. Comandos Slash Principales

- `/grill-me` : Entrevista interactiva donde el agente pregunta detalladamente al usuario para acordar un plan de arquitectura antes de tocar el código.
- `/goal` : Activa un modo de ejecución profunda donde el agente persiste y no se detiene hasta completar objetivos complejos de larga duración.
- `/schedule` : Configura temporizadores o tareas recurrentes.
- `/browser` : Invoca capacidades avanzadas de automatización y scraping web.
- `/learn` : Registra aprendizajes y soluciones a problemas complejos en el repositorio para futuras sesiones.

5.2. Menú de Menciones (@)

Al escribir `@` en el chat, se despliega el menú contextual para adjuntar:

- **Archivos y Directorios:** `@src/bixpe_bot.py`
- **Conversaciones Previas:** `@Conversation-ID`
- **Sesiones de Terminal:** `@Terminal-1`
- **Servidores y Herramientas MCP:** `@mcp-server`

6. Seguridad, Sandbox y Control de Permisos

Para proteger entornos de producción, Antigravity 2.5.0 implementa una arquitectura de seguridad configurable:

Regla de Seguridad Fundamental: Las credenciales, tokens de API o contraseñas NUNCA deben escribirse en texto plano en los archivos de código. Se deben utilizar siempre variables de entorno (`.env`) o secretos de repositorio (`GitHub Secrets`).

Políticas de Ejecución de Comandos

- `always-proceed` : Ejecución automática de comandos (para entornos de desarrollo rápido).
- `request-review` : El agente pide confirmación previa al usuario antes de ejecutar cualquier comando.
- `proceed-in-sandbox` : Ejecución aislada dentro de un contenedor seguro.

7. Guía de Integración Práctica y Casos de Uso

A continuación se resume el flujo recomendado para implementar mejoras en sistemas en producción (como la automatización de notificaciones duales Telegram/Email):

1. **Aislamiento en Rama:** Crear una rama secundaria (ej. `feature/alertas-telegram`) y prohibir commits directos en `main` .
2. **Reforzado de Excepciones:** Implementar estructuras `try/except` en Python para garantizar que cualquier fallo del bot llame a `sys.exit(1)` .
3. **Alertas en GitHub Actions:** Configurar pasos condicionales `if: failure()` en los archivos de flujo `.yaml` llamando a la API de Telegram con los secretos `TELEGRAM_TOKEN` y `TELEGRAM_CHAT_ID` .
4. **Revisión mediante Pull Request:** Abrir un PR hacia `main` para poder probar en modo simulación antes de fusionar.

— Fin del Manual Oficial de Google Antigravity 2.5.0 —